МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«ОРГАНІЗАЦІЯ БАЗ ДАНИХ»

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 6

Виконав:
студент групи КН-25-1
Рафієв Б. С.
Перевірив:
асистент кафедри АІС
Андреєв П.І.

Кременчук 2026

Тема: **Представлення.**

Мета: навчитися створювати та використовувати представлення.

Порядок виконання роботи

Лабораторну роботу виконано в середовищі Microsoft SQL Server Management Studio 22. Для першої частини використано навчальну базу даних Sales111, створену під час лабораторної роботи № 2; для індивідуального завдання – базу даних LibraryDB (варіант № 1 «Бібліотека»). Представлення створено командою CREATE VIEW, для контролю значень застосовано параметр WITH CHECK OPTION. Результати виконання наведено на рисунках. Після зняття скріншотів змінені під час демонстрації дані відновлено до початкового стану.

1. Представлення для бази даних Sales111

У цій частині створено п’ять представлень для бази даних Sales111: TopCustomers, SellersByCity, Commissions, OrdersWithNames і AboveAvgOrders. Для кожного представлення наведено код створення та результат звернення до нього. Окремо перевірено дію параметра WITH CHECK OPTION та проаналізовано модифікованість чотирьох представлень із методичних вказівок.

1.1. Створити представлення, яке показує всіх замовників з найвищими рейтингами.

```
CREATE VIEW TopCustomers AS
SELECT ID_CM AS Код_замовника,
       NAME_CM AS Замовник,
       CITY_CM AS Місто,
       BONUS_CM AS Рейтинг
FROM Customers
WHERE BONUS_CM = (SELECT MAX(BONUS_CM) FROM Customers);
GO

SELECT * FROM TopCustomers;
```

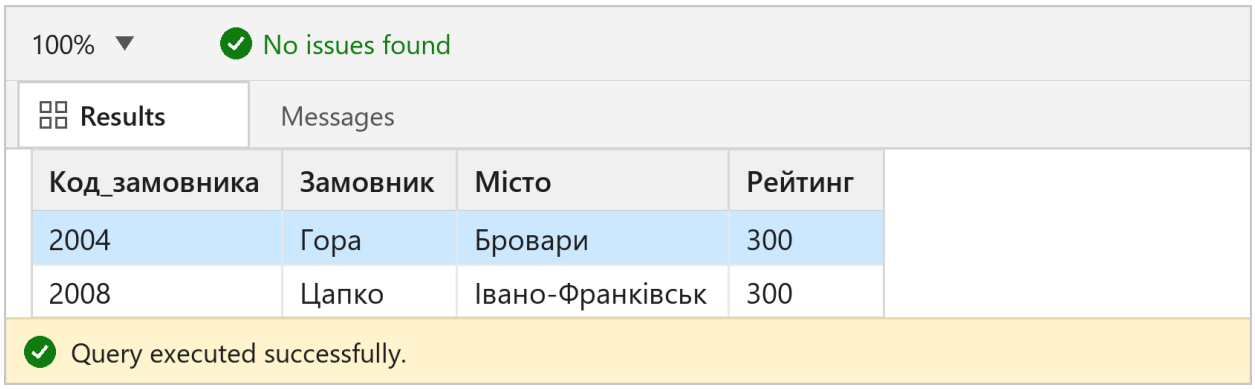


Рисунок 1 – Результат виконання запиту 1.1

Представлення побудоване на одній таблиці, а максимальний рейтинг визначається підзапитом. До результату потрапили замовники з рейтингом 300 – Гора та Цапко.

1.2. Створити представлення, яке показує кількість продавців у кожному місті.

```
CREATE VIEW SellersByCity AS
SELECT CITY_SP AS Місто,
       COUNT(*) AS Кількість_продавців
FROM Salespeople
GROUP BY CITY_SP;
GO

SELECT * FROM SellersByCity;
```

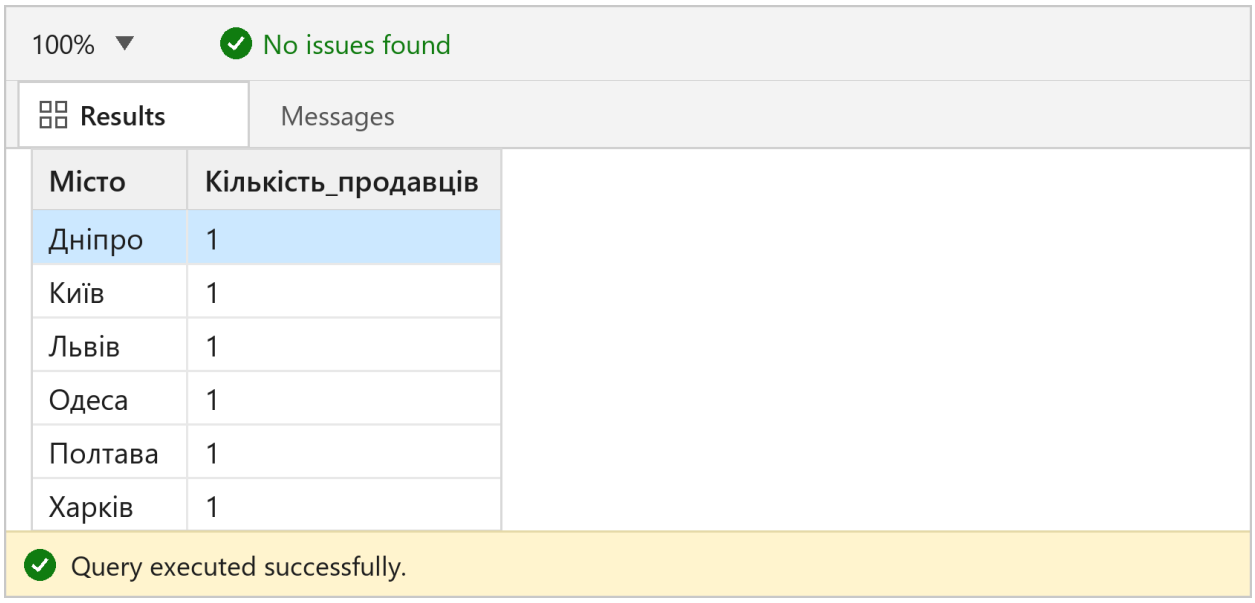


Рисунок 2 – Результат виконання запиту 1.2

Представлення містить агрегатну функцію та GROUP BY, тому є немодифікованим: через нього не можна виконувати INSERT, UPDATE або DELETE.

1.3.1. Створити представлення Commissions (Комісійні), яке включає тільки поля COMM_SP і ID_SP і дозволяє змінювати комісійні в межах від 0,10 до 0,20.

```
CREATE VIEW Commissions AS
SELECT ID_SP, COMM_SP
FROM Salespeople
WHERE COMM_SP BETWEEN 0.10 AND 0.20
WITH CHECK OPTION;
GO

SELECT * FROM Commissions;
```

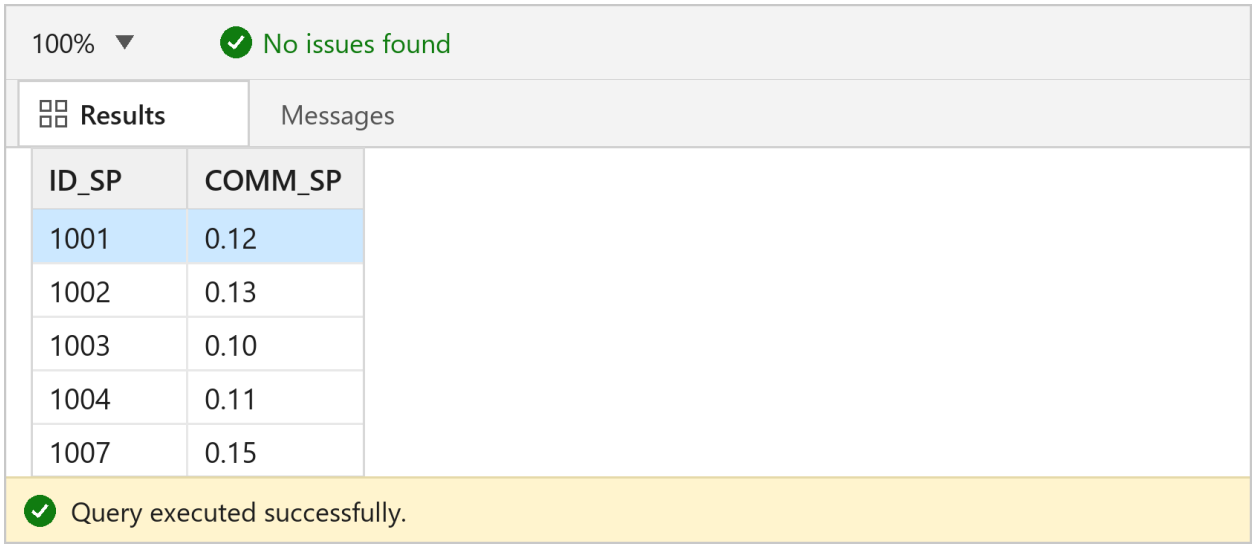


Рисунок 3 – Результат виконання запиту 1.3.1

Умова WHERE разом із WITH CHECK OPTION дозволяє вводити або змінювати комісійні лише в діапазоні 0,10–0,20. Продавець 1005 із комісійними 0,09 до представлення не потрапляє.

1.3.2. Змінити комісійні продавця 1003 на 0,14 через представлення Commissions.

```
UPDATE Commissions
SET COMM_SP = 0.14
WHERE ID_SP = 1003;

SELECT * FROM Commissions;
```

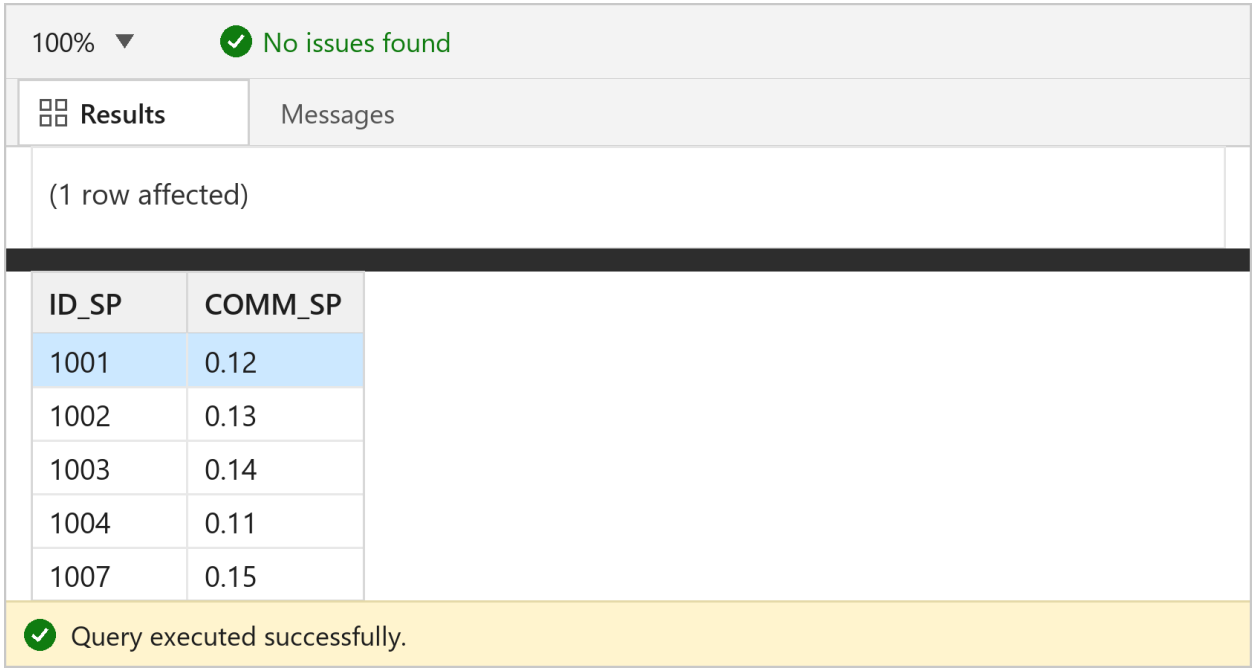


Рисунок 4 – Результат виконання запиту 1.3.2

Значення 0,14 входить до діапазону 0,10–0,20, тому команда UPDATE виконується успішно. Зміна одразу відображається в базовій таблиці Salespeople.

1.3.3. Перевірити дію WITH CHECK OPTION: спроба встановити комісійні 0,25.

```
UPDATE Commissions
SET COMM_SP = 0.25
WHERE ID_SP = 1003;
```

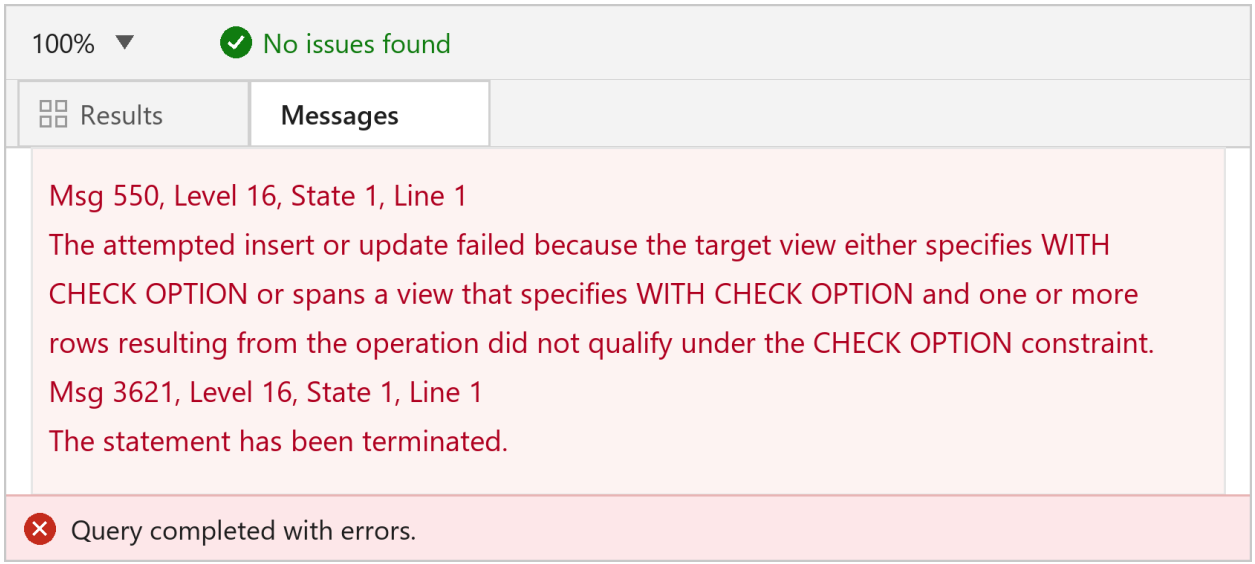


Рисунок 5 – Результат виконання запиту 1.3.3

Значення 0,25 виходить за межі діапазону 0,10–0,20, тому сервер повертає помилку 550 і скасовує зміну. Це підтверджує, що WITH CHECK OPTION унеможливлює порушення умови представлення.

1.4. Створити представлення з прізвищами замовників і продавців для кожного замовлення та вивести замовлення клієнта Гора.

```
CREATE VIEW OrdersWithNames AS
SELECT o.ID_OD AS Код_замовлення,
       c.NAME_CM AS Замовник,
       s.NAME_SP AS Продавець,
       o.AMOUNT_OD AS Сума,
       o.DATE_OD AS Дата
FROM Orders o
JOIN Customers c ON c.ID_CM = o.ID_CM
JOIN Salespeople s ON s.ID_SP = o.ID_SP;
GO

SELECT * FROM OrdersWithNames
WHERE Замовник = N'Гора';
```

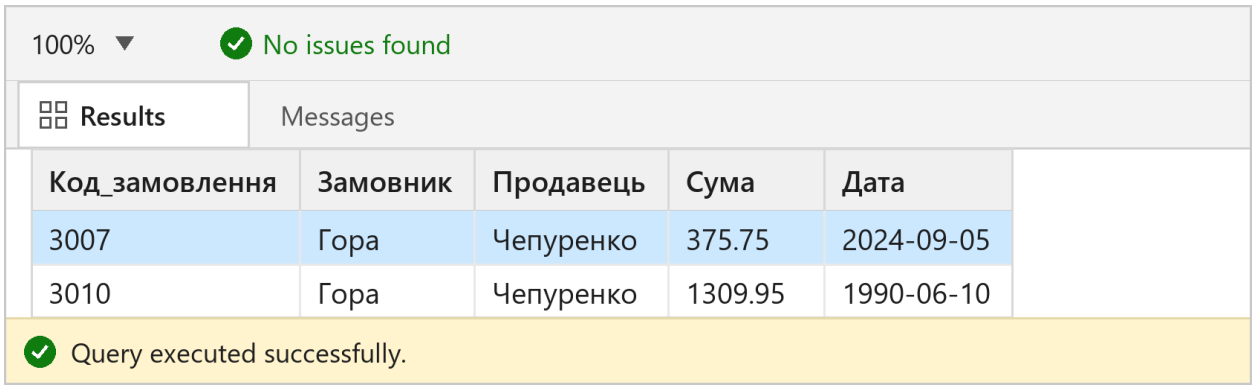


Рисунок 6 – Результат виконання запиту 1.4

Представлення об’єднує три таблиці, тому є немодифікованим. Умову відбору за прізвищем застосовано до представлення так само, як і до звичайної таблиці.

1.5. Створити представлення, яке показує всі замовлення із сумами, вищими за середню.

```
CREATE VIEW AboveAvgOrders AS
SELECT ID_OD AS Код_замовлення,
       DATE_OD AS Дата,
       AMOUNT_OD AS Сума,
       ID_CM AS Код_замовника,
       ID_SP AS Код_продавця
FROM Orders
WHERE AMOUNT_OD > (SELECT AVG(AMOUNT_OD) FROM Orders);
GO

SELECT * FROM AboveAvgOrders
ORDER BY Сума DESC;
```

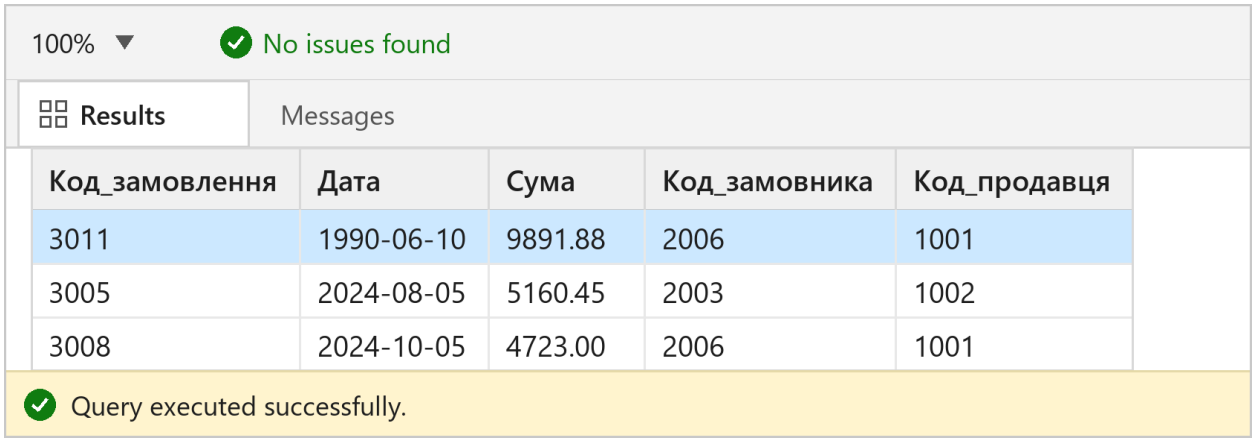


Рисунок 7 – Результат виконання запиту 1.5

Середня сума замовлення становить 2745,84 грн, тому до представлення потрапили три замовлення. Підзапит в умові WHERE не робить представлення незмінюваним: воно побудоване на одній таблиці.

1.6. Які з наведених нижче представлень будуть модифікованими і чому?

Для відповіді на питання проаналізуємо кожне представлення за критеріями модифікованості: одна базова таблиця, наявність первинного ключа, відсутність DISTINCT, GROUP BY, HAVING, агрегатних функцій, UNION і обчислюваних стовпців.

Представлення	Модифіковане	Причина
Dailyorders	Ні	Містить ключове слово DISTINCT, тому представлення не є модифікованим.
Custtotals	Ні	Містить агрегатну функцію SUM і GROUP BY, а також будується на двох таблицях; такі представлення доступні лише для читання.
Thirdorders	Ні	Побудоване на представленні Dailyorders, яке не є модифікованим; похідне представлення успадковує цю властивість.

Представлення	Модифіковане	Причина
Nullcities	Так	Побудоване на одній таблиці Salespeople, не містить DISTINCT, GROUP BY, агрегатів і обчислюваних стовпців; умова WHERE не перешкоджає модифікації.

У коді представлення Custtotals з методичних вказівок є друкарська помилка: Customer.id_cm; правильно – Customers.id_cm.

2. Індивідуальне завдання. База даних LibraryDB

Індивідуальне завдання виконано для варіанта № 1 – предметна область «Бібліотека». Використано створену в лабораторній роботі № 2 базу даних LibraryDB, що містить таблиці Author, Publisher, Book, Reader і BookLoan. Створено чотири модифіковані представлення та виконано через них команди INSERT, UPDATE і DELETE.

2.1. Створити модифіковане представлення BooksCheap для книг ціною менше 400 грн.

```
CREATE VIEW BooksCheap AS
SELECT id_book, title, id_author, id_publisher, year_pub, price
FROM Book
WHERE price < 400
WITH CHECK OPTION;
GO

SELECT * FROM BooksCheap;
```

100% ▼ ✓ No issues found

Results | Messages

id_book	title	id_author	id_publisher	year_pub	price
1	Кобзар	1	1	2021	350.00
2	Захар Беркут	2	2	2019	280.00
3	Лісова пісня	3	1	2020	220.00

✓ Query executed successfully.

Рисунок 8 – Результат виконання запиту 2.1

Представлення побудоване на одній базовій таблиці Book, містить первинний ключ id_book і не використовує DISTINCT, GROUP BY чи агрегати, тому є модифікованим.

2.2. Створити модифіковане представлення BooksModern для книг, виданих починаючи з 2020 року.

```
CREATE VIEW BooksModern AS
SELECT id_book, title, id_author, id_publisher, year_pub, price
FROM Book
WHERE year_pub >= 2020
WITH CHECK OPTION;
GO

SELECT * FROM BooksModern;
```

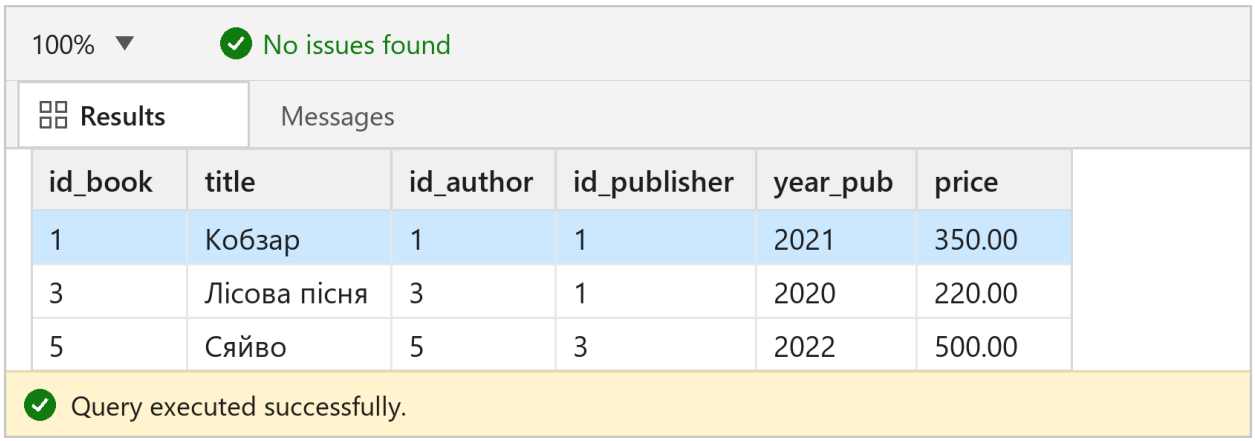


Рисунок 9 – Результат виконання запиту 2.2

Завдяки WITH CHECK OPTION через це представлення не можна змінити рік видання на такий, що не задовольняє умову year_pub >= 2020.

2.3. Створити модифіковане представлення LoansReturned для повернутих книг.

```
CREATE VIEW LoansReturned AS
SELECT id_loan, id_book, id_reader, loan_date, return_date
FROM BookLoan
WHERE return_date IS NOT NULL;
GO

SELECT * FROM LoansReturned;
```

100% ▼ ✓ No issues found

Results | Messages

id_loan	id_book	id_reader	loan_date	return_date
1	1	1	2026-09-01	2026-09-15
3	3	2	2026-09-03	2026-09-18
5	5	3	2026-09-10	2026-09-20
7	2	4	2026-09-15	2026-09-25
9	4	5	2026-09-18	2026-09-27

✓ Query executed successfully.

Рисунок 10 – Результат виконання запиту 2.3

Представлення містить первинний ключ id_loan; умова IS NOT NULL не перешкоджає модифікації, оскільки не використовує агрегатів чи групування.

2.4. Створити модифіковане представлення LoansOpen для неповернутих книг.

```
CREATE VIEW LoansOpen AS
SELECT id_loan, id_book, id_reader, loan_date, return_date
FROM BookLoan
WHERE return_date IS NULL
WITH CHECK OPTION;
GO

SELECT * FROM LoansOpen;
```

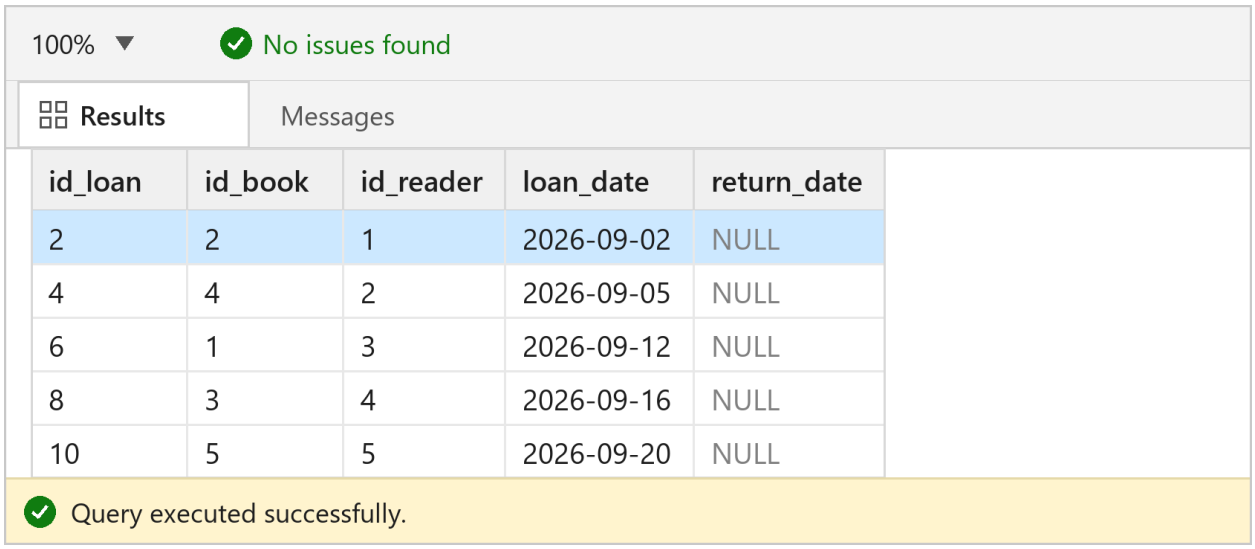


Рисунок 11 – Результат виконання запиту 2.4

WITH CHECK OPTION забороняє встановлювати дату повернення через це представлення, оскільки рядок перестав би задовольняти умову return_date IS NULL.

2.5. Додати нову книгу через представлення BooksCheap (команда INSERT).

```
INSERT INTO BooksCheap (title, id_author, id_publisher, year_pub, price)
VALUES (N'Тіні забутих предків', 2, 2, 2023, 300);

SELECT * FROM BooksCheap;
```

100% ▼ No issues found

Results | Messages

(1 row affected)

id_book	title	id_author	id_publisher	year_pub	price
1	Кобзар	1	1	2021	350.00
2	Захар Беркут	2	2	2019	280.00
3	Лісова пісня	3	1	2020	220.00
6	Тіні забутих предків	2	2	2023	300.00

Query executed successfully.

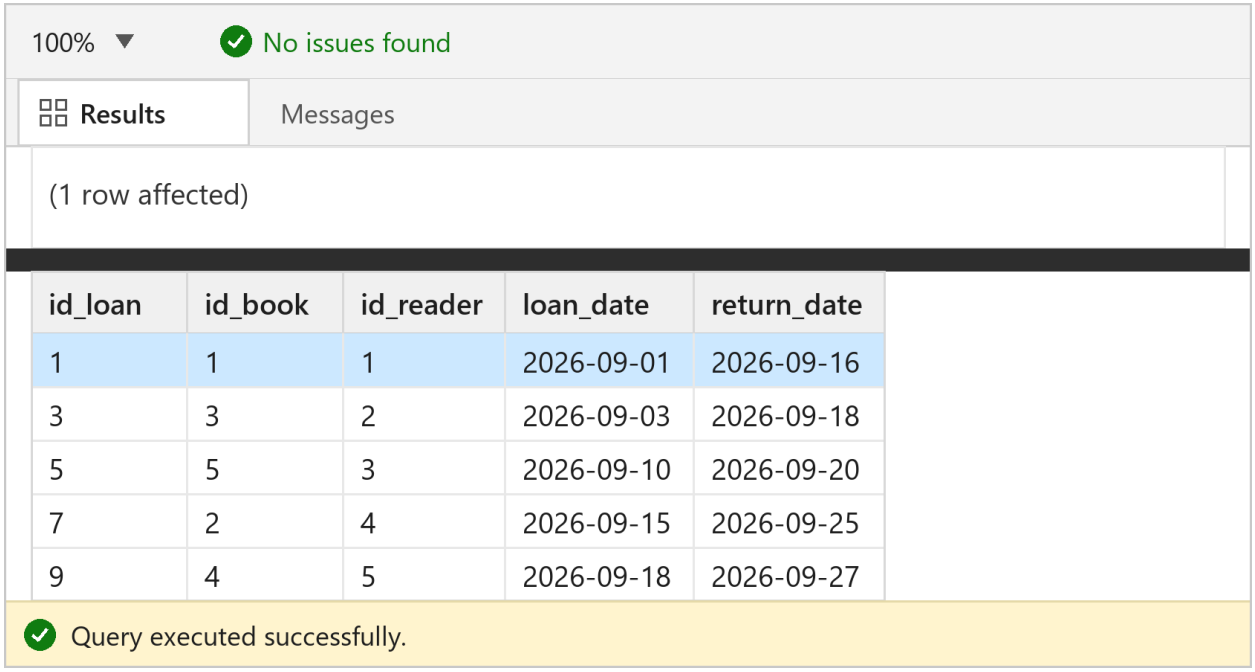
Рисунок 12 – Результат виконання запиту 2.5

Команда INSERT через представлення додає рядок до базової таблиці Book; ціна 300 грн задовольняє умову представлення, тому WITH CHECK OPTION не перешкоджає вставці.

2.6. Змінити дату повернення книги через представлення LoansReturned (команда UPDATE).

```
UPDATE LoansReturned
SET return_date = '2026-09-16'
WHERE id_loan = 1;

SELECT * FROM LoansReturned;
```



id_loan	id_book	id_reader	loan_date	return_date
1	1	1	2026-09-01	2026-09-16
3	3	2	2026-09-03	2026-09-18
5	5	3	2026-09-10	2026-09-20
7	2	4	2026-09-15	2026-09-25
9	4	5	2026-09-18	2026-09-27

Рисунок 13 – Результат виконання запиту 2.6

Оновлено дату повернення для видачі № 1; зміна відображена як у представленні, так і в базовій таблиці BookLoan.

2.7. Перевірити дію WITH CHECK OPTION у представленні LoansOpen.

```
UPDATE LoansOpen
SET return_date = '2026-10-01'
WHERE id_loan = 4;
```

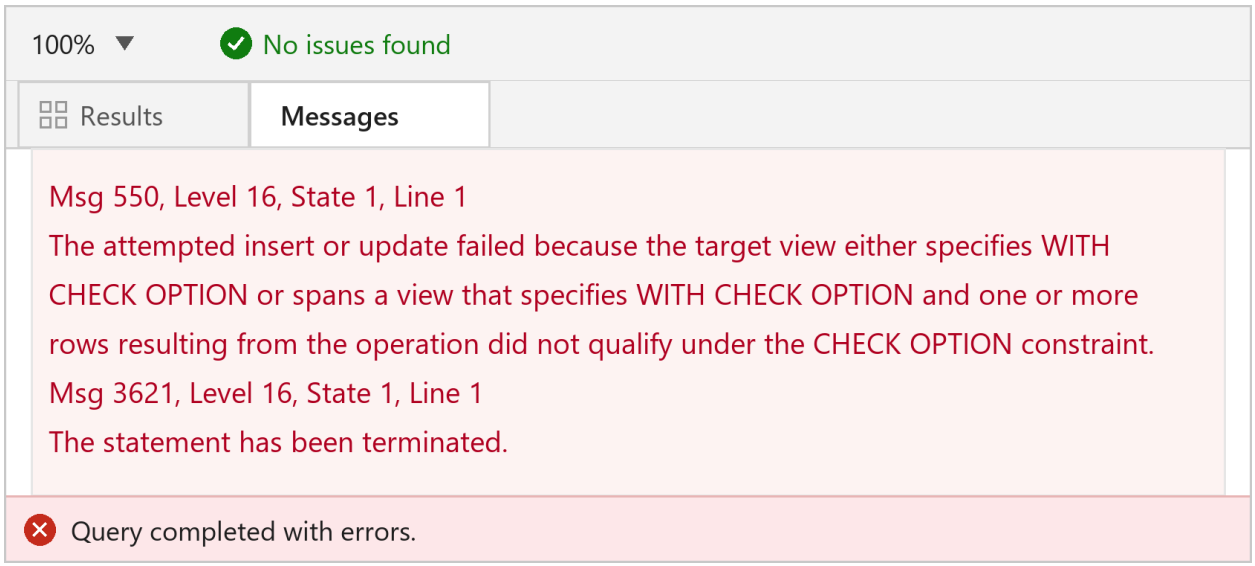


Рисунок 14 – Результат виконання запиту 2.7

Спроба встановити дату повернення через представлення неповернутих книг порушує умову return_date IS NULL, тому сервер повертає помилку 550 і скасовує зміну.

2.8. Видалити видачу через представлення LoansOpen (команда DELETE).

```
DELETE FROM LoansOpen
WHERE id_loan = 2;

SELECT * FROM LoansOpen;
```

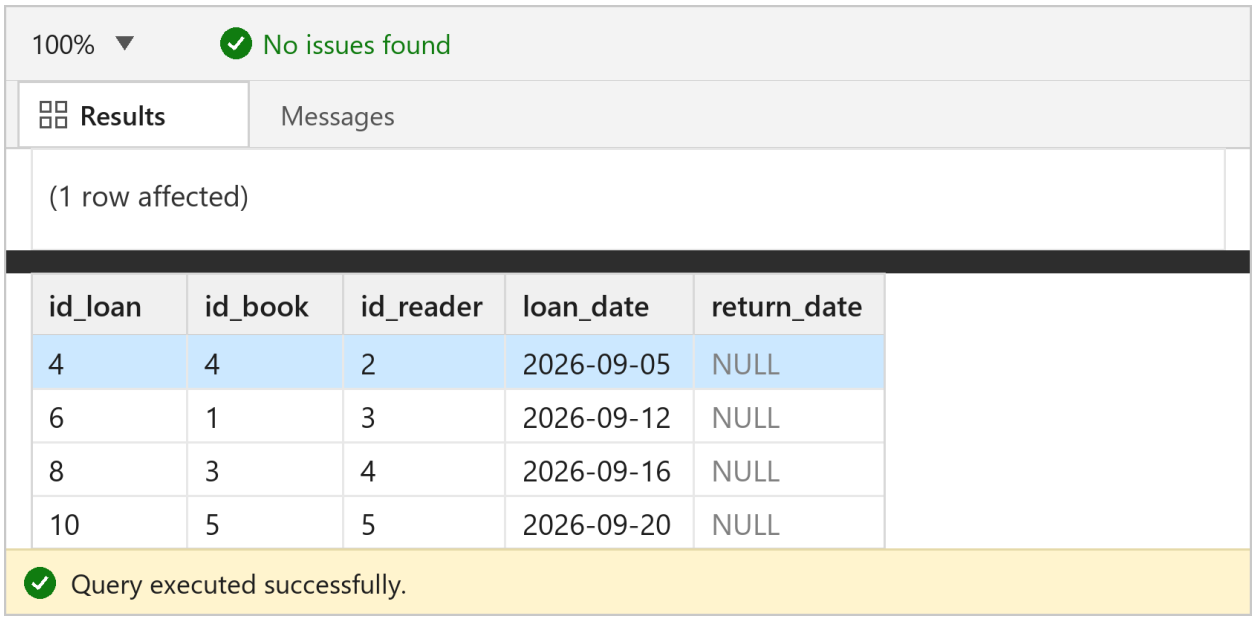


Рисунок 15 – Результат виконання запиту 2.8

Рядок видаляється з базової таблиці BookLoan; після виконання команди у представленні залишилося чотири неповернуті видачі.

Відповіді на контрольні питання

1. Що таке представлення? Для чого використовують представлення?

Представлення (view) – це збережений у базі даних іменований запит, який для користувача виглядає як звичайна таблиця. Саме представлення не містить даних (за винятком індексованих представлень): під час звернення до нього сервер виконує збережений оператор SELECT і повертає актуальні рядки базових таблиць.

Представлення використовують, щоб:

1) спростити роботу зі складними запитами – один раз описати з'єднання кількох таблиць і надалі звертатися до результату як до однієї таблиці;

2) обмежити доступ до даних – показати користувачу лише потрібні стовпці або рядки, не надаючи прав безпосередньо на базові таблиці;

3) забезпечити логічну незалежність програм від структури таблиць: зміни в таблицях можна приховати за незмінним інтерфейсом представлення;

4) стандартизувати доступ – кілька застосунків використовують однакове представлення;

5) готувати дані для звітів і аналізу.

Приклад із цього звіту – представлення OrdersWithNames (запит 1.4), яке приховує з'єднання трьох таблиць:

```
SELECT * FROM OrdersWithNames
WHERE Замовник = N'Гора';
```

Звернення до представлення виглядає так само, як і до звичайної таблиці, але щоразу виконується збережений запит до базових таблиць.

2. Охарактеризуйте типи представлень і їх особливості.

За складністю розрізняють прості та складні представлення. Просте побудоване на одній базовій таблиці й не містить агрегатів, групування чи об'єднань; складне може поєднувати кілька таблиць, підзапити, UNION і агрегатні функції.

За можливістю змінювати дані представлення поділяють на модифіковані (updatable) і немодифіковані (read-only). Через модифіковане представлення можна виконувати INSERT, UPDATE і DELETE; немодифіковане призначене лише для читання.

За додатковими властивостями виділяють:

1) представлення з WITH CHECK OPTION – контролює, щоб вставлені та змінені рядки задовольняли умову представлення;

2) вкладені представлення – побудовані на інших представленнях;

3) індексовані (матеріалізовані) – мають унікальний кластерний індекс і фізично зберігають результат запиту;

4) секціоновані – об’єднують дані кількох таблиць, розподілених за ключем;

5) системні – службові представлення каталогу (sys.objects, sys.columns тощо).

У цій роботі використано прості модифіковані представлення (BooksCheap, BooksModern, LoansReturned, LoansOpen), представлення з WITH CHECK OPTION (Commissions, BooksCheap, BooksModern, LoansOpen) і немодифіковані представлення (SellersByCity, OrdersWithNames).

3. Як здійснюється створення і видалення представлень?

Представлення створюють командою CREATE VIEW, після якої записують назву та один оператор SELECT:

```
CREATE VIEW BooksCheap AS
SELECT id_book, title, id_author, id_publisher, year_pub, price
FROM Book
WHERE price < 400
WITH CHECK OPTION;
```

Якщо потрібно задати власні назви стовпців, їх перелічують у дужках після назви представлення. Команда CREATE VIEW має бути першим оператором пакета, тому в скрипті перед нею ставлять GO.

Для зміни визначення використовують ALTER VIEW, для видалення – DROP VIEW:

```
ALTER VIEW BooksCheap AS
SELECT id_book, title, price FROM Book WHERE price < 500;

DROP VIEW IF EXISTS BooksCheap;
```

У MS SQL Server немає конструкції CREATE OR REPLACE VIEW, поширеної в Oracle і PostgreSQL: замість неї застосовують ALTER VIEW або пару DROP VIEW + CREATE VIEW. Перейменувати представлення можна процедурою sp_rename. Для створення потрібні право CREATE VIEW у базі даних і право SELECT на базові таблиці.

4. Опишіть використання команд модифікації в представленні.

Якщо представлення модифіковане, через нього виконують звичайні команди INSERT, UPDATE і DELETE. Сервер автоматично перетворює їх на відповідні операції над базовою таблицею, тому зміни зберігаються саме в таблиці, а представлення лише показує їх.

У цьому звіті через представлення виконано всі три команди:

```
INSERT INTO BooksCheap (title, id_author, id_publisher, year_pub, price)
VALUES (N'Тіні забутих предків', 2, 2, 2023, 300);

UPDATE LoansReturned
SET return_date = '2026-09-16'
WHERE id_loan = 1;

DELETE FROM LoansOpen
WHERE id_loan = 2;
```

Особливості: не можна змінювати обчислювані стовпці; під час INSERT потрібно заповнити всі стовпці базової таблиці, які не мають значення за замовчуванням і не допускають NULL; якщо представлення містить WITH CHECK OPTION, сервер перевіряє кожен змінений рядок. Для складних випадків (наприклад, представлення на кількох таблицях) застосовують тригери INSTEAD OF.

5. Назвіть критерії представлення, що модифікується.

Представлення є модифікованим, якщо виконуються такі умови:

1) визначення містить лише одну базову таблицю (без JOIN, UNION і підзапитів у списку стовпців);

2) немає DISTINCT, GROUP BY, HAVING і агрегатних функцій;

3) немає обчислюваних стовпців, які потрібно змінювати;

4) представлення містить первинний ключ базової таблиці, що гарантує однозначне визначення рядка;

5) для INSERT у представленні наявні всі стовпці базової таблиці, які не допускають NULL і не мають значення за замовчуванням.

Саме тому Nullcities з питання 1.6 є модифікованим, а Dailyorders (DISTINCT), Custtotals (GROUP BY, SUM, дві таблиці) і Thirdorders (побудоване на немодифікованому представленні) – ні.

6. Як здійснюється перевірка значень, що поміщаються у представлення?

Для контролю значень, що вставляються або змінюються через представлення, використовують параметр WITH CHECK OPTION. Він змушує сервер перевіряти кожен новий або змінений рядок на відповідність умові WHERE представлення. Якщо рядок не задовольняє умову, операція скасовується з помилкою 550.

Приклад – представлення Commissions (запит 1.3.1):

```
CREATE VIEW Commissions AS
SELECT ID_SP, COMM_SP
FROM Salespeople
WHERE COMM_SP BETWEEN 0.10 AND 0.20
WITH CHECK OPTION;
```

Команда UPDATE Commissions SET COMM_SP = 0.25 ... завершується помилкою, а значення 0,14 встановлюється успішно. Аналогічно працює представлення LoansOpen: спроба встановити дату повернення порушує умову return_date IS NULL і скасовується (запит 2.7). Крім WITH CHECK OPTION, значення контролюють обмеження CHECK, зовнішні ключі, тригери та права доступу.

Висновки:

Під час виконання лабораторної роботи № 6 було опановано створення та використання представлень у MS SQL Server. Розглянуто команди CREATE VIEW, ALTER VIEW і DROP VIEW, параметр WITH CHECK OPTION, а також особливості модифікованих і немодифікованих представлень. Для бази даних Sales111 створено п’ять представлень: TopCustomers, SellersByCity, Commissions, OrdersWithNames і AboveAvgOrders. На прикладі представлення Commissions показано, що WITH CHECK OPTION дозволяє змінювати комісійні лише в межах 0,10–0,20 і скасовує спробу встановити значення 0,25 з помилкою 550. Для бази даних LibraryDB (варіант № 1 «Бібліотека») створено чотири модифіковані представлення – BooksCheap, BooksModern, LoansReturned і LoansOpen – та виконано через них команди INSERT, UPDATE і DELETE. Окремо проаналізовано чотири представлення з методичних вказівок і встановлено, що модифікованим є лише Nullcities. Отримані результати підтверджують, що представлення спрощують доступ до даних, приховують складність запитів і дозволяють контролювати зміни на рівні сервера. Надано відповіді на всі шість контрольних питань.